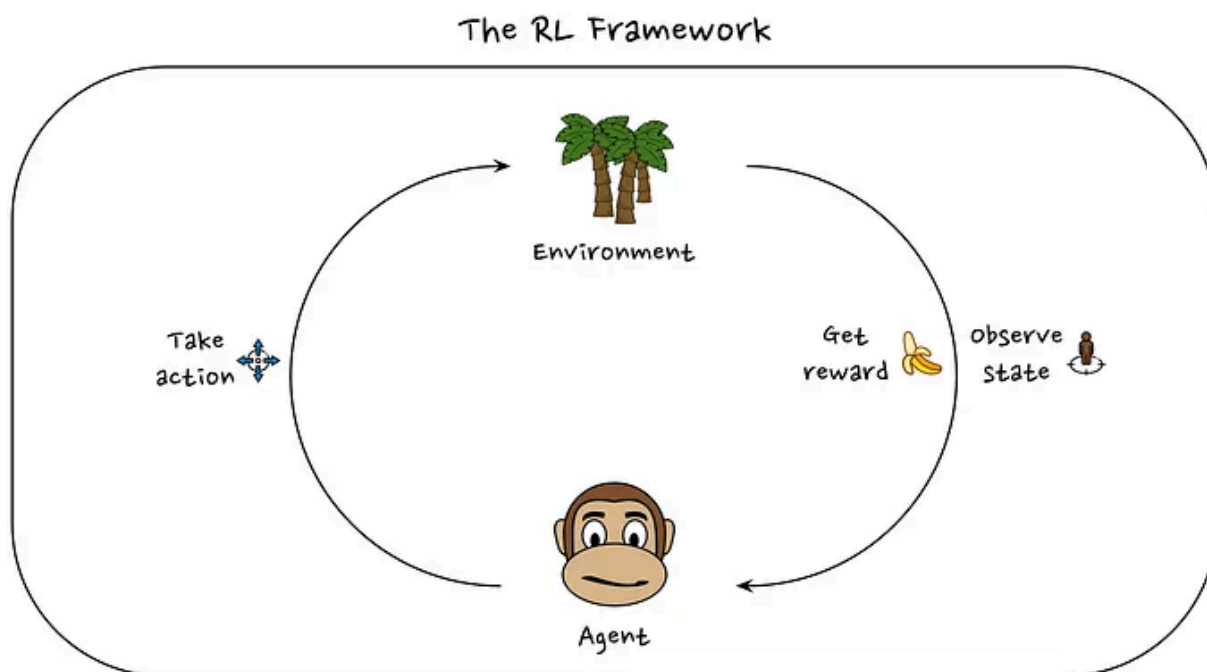


Laboratório 5 — “Ensinando um agente a aprender”

Agora que você já está familiarizada com as funções básicas do Gymnasium, sabe implementar os ambientes — e de quebra ainda aumentou as suas habilidades com jogos de cartas —, podemos dar um passo à frente nas políticas que irão guiar o comportamento do agente.

Vimos que as políticas fixas ajudam a melhorar as recompensas do comportamento do agente quando comparadas à decisões de ações aleatórias, as quais sequer levam em consideração o estado atual do agente. Porém, elas ainda são bastante ineficientes, haja vista que apenas aplicam uma regra — ou um conjunto de regras — imutável e não levam em conta as observações anteriores do ambiente (o seu contexto).

Por isso, vamos começar a explorar técnicas que dão um *upgrade* nas políticas dos agentes. Ao invés de determiná-las nós mesmos, vamos deixar que o agente aprenda através da experiência como devem ser as regras que maximizam a pontuação que pode ser obtida naquele ambiente. Esse é o princípio do Aprendizado por Reforço (no inglês *Reinforcement Learning*). Nas últimas semanas, nós já estávamos lidando com sua estrutura básica. Podemos lembrá-la observando a figura abaixo:



Fonte: <https://medium.com/@cedric.vandelaer/reinforcement-learning-an-introduction-part-1-4-866695deb4d1>

Imagine que o agente é um macaco e a tarefa a ser resolvida é coletar a maior quantidade possível de bananas. A cada instante de tempo o macaco decide fazer uma ação. As ações podem ser, por exemplo, andar em direção a uma árvore, agarrar alguma coisa, escalar, etc. Vamos definir a recompensa a cada instante de tempo como o número de bananas que o macaco ganha. Depois de cada ação, o macaco estará em um novo estado, definido arbitrariamente por nós como a sua posição no mundo. Então, quando um macaco realiza uma nova ação (*step*), o estado no próximo

instante de tempo será as suas novas coordenadas nesse instante. O que está sendo buscado aqui é o comportamento ótimo, ou seja, a melhor sequência de ações que macaco pode realizar para maximizar a quantidade de bananas que conseguirá.

Para descobrir a sequência ótima de ações, o macaco não depende apenas do estado atual, mas também de suas experiências passadas. Cada tentativa gera informação sobre quais escolhas trouxeram mais bananas ao longo do tempo. Com isso, o agente atualiza suas estimativas de valor das ações e aprende gradualmente uma regra de comportamento que o leva a maximizar a recompensa acumulada. Assim, experiências anteriores guiam as próximas decisões, permitindo que o macaco evolua de escolhas aleatórias para um comportamento eficiente.

Q-Learning

Uma das técnicas de RL é chamada Q-Learning. Nela, os agentes aprendem por tentativa e erro e tomarão as decisões conforme o peso de cada uma das possíveis novas posições/ estados. A ação que tiver maior valor será a escolhida para guiar o estado. A sequência de passos desta técnica é ilustrada abaixo.



Fonte: <https://medium.com/swlh/introduction-to-q-learning-with-openai-gym-2d794da10f3d>

Para calcular os valores das posições possíveis do tabuleiro, a ideia é construir uma *look up table* conforme a seguinte equação:

$$Q^{new}(S_t, A_t) \leftarrow (1 - \underbrace{\alpha}_{\text{learning rate}}) \cdot \underbrace{Q(S_t, A_t)}_{\text{current value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \underbrace{\left(\underbrace{R_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(S_{t+1}, a)}_{\text{estimate of optimal future value}} \right)}_{\text{new value (temporal difference target)}}$$

- R_{t+1} : recompensa recebida quando se move de um estado S_t para outro S_{t+1}
- α : learning rate tal que $0 < \alpha \leq 1$
- αR_{t+1} : recompensa R_{t+1} se a ação A_t é tomada quando está em um estado S_t ponderado pela learning rate
- $\alpha \gamma \max_a Q(S_{t+1}, a)$: a recompensa máxima que pode ser obtida de um estado S_{t+1} ponderada pela learning rate e por um fator de desconto
- $\max_a Q(S_{t+1}, a)$: olha para a linha do novo estado S_{t+1} na Q-Table e escolhe a ação com maior pontuação de valor

A **learning rate** (α): determina o quanto as novas informações adquiridas sobrescrevem as informações antigas.

- 0: nenhum aprendizado, ou seja, está agente está apenas explorando conhecimentos à priori
- 1: faz o agente considerar apenas as informações mais recentes, ou seja, o conhecimento à priori é ignorado para poder explorar as possibilidades

O **discount factor** (γ): determina a importância das recompensas futuras.

- 0: agente considera apenas as recompensas atuais
- 1: agente irá se esforçar para as recompensas a longo prazo

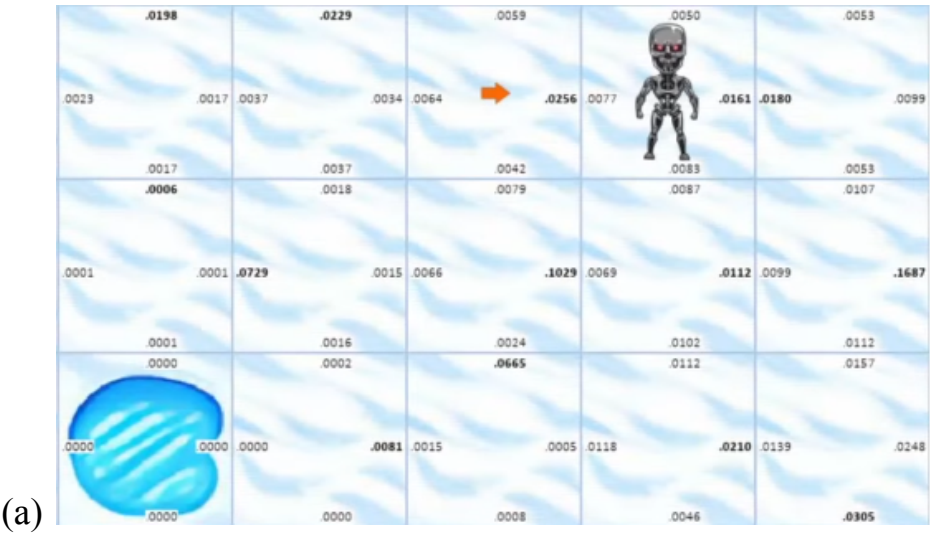
Nós usamos a equação para calcular cada uma das combinações entre linha e coluna da tabela. Depois do treinamento, essa tabela estará completa e o agente irá escolher a próxima casa do tabuleiro com base na maior pontuação dentre os possíveis novos locais (dada a posição atual).

Tabela 1: exemplo de Q-Table de um agente treinado em um tabuleiro 7 x 7 de Frozen Lake.

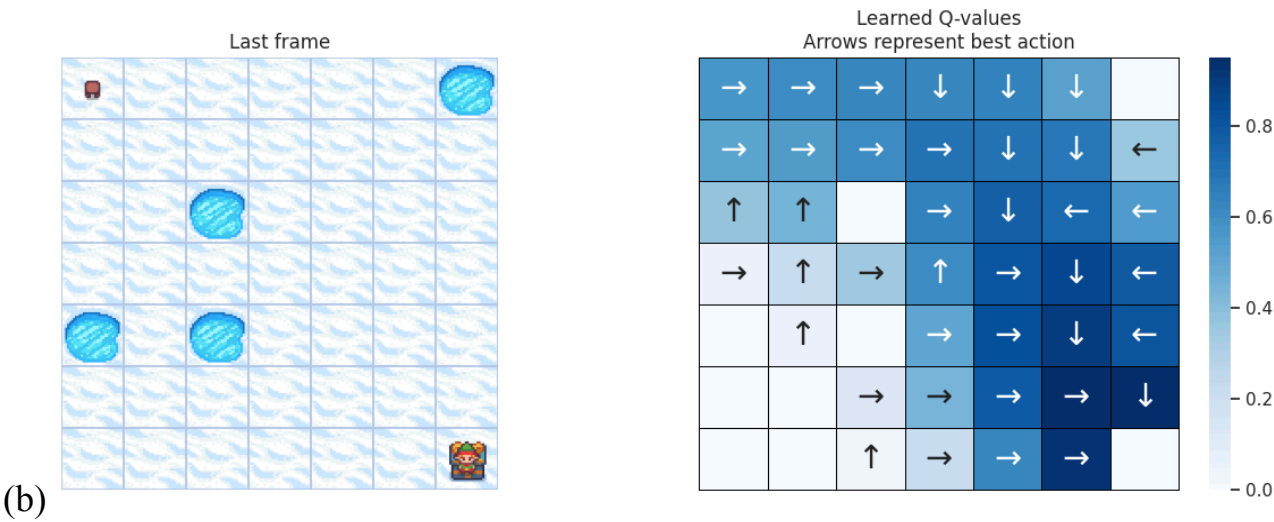
		Ações			
		0 (esquerda)	1 (baixo)	2 (direita)	3 (cima)
Estados	0	0,60	0,25	0,10	0,05
	1	0,75	0,15	0,05	0,05
	2	0,65	0,20	0,05	0,10
	3	0,20	0,70	0,15	0,15
	...	...	...	...	...
	6	0	0	0	0
	...	...	...	...	...
	47	0,90	0,01	0,02	0,07
	48	0	0	0	0

Abaixo podemos ver alguns exemplos reais dos scores de cada posição (estado) do tabuleiro em relação às possíveis ações no início de um treinamento (a). Também há uma representação mais visual das informações da Tabela 1, em que as setas representam a melhor ação em cada estado, ou seja, aquela que irá maximizar as recompensas a longo prazo (b). E por fim, em (c) temos um gráfico das recompensas associadas a cada episódio ao longo das iterações.

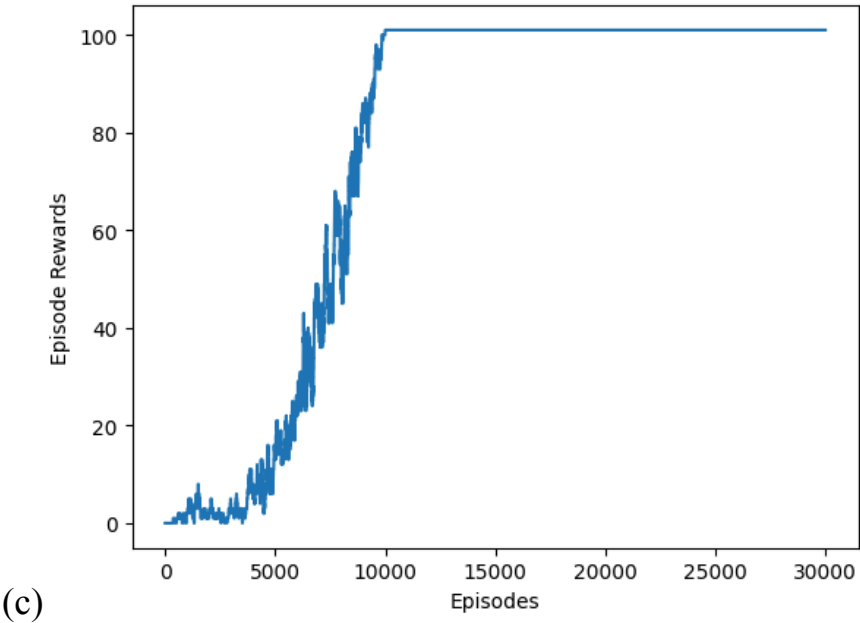
Introdução ao Gymnasium e Agentes com Políticas Aprendidas



Fonte: <https://www.youtube.com/watch?v=ZhoIgo3qqLU>



Fonte: https://gymnasium.farama.org/tutorials/training_agents/frozenlake_q_learning/



Fonte: <https://www.youtube.com/watch?v=ZhoIgo3qqLU>

Para implementar a nossa política de agente, vamos combinar a Q-Table com o Epsilon-Greedy Algorithm. Isso significa que no começo as ações desempenhadas pelo agente são aleatórias, mas passarão a ser cada vez mais ditadas pelas regras da Q-Table ao longo dos episódios. Após cada episódio, iremos reduzir o epsilon até que ele seja zero (em algum momento).

Ufa! Depois desse bombardeio teórico, mãos à obra!

Diferente dos laboratórios anteriores, neste serão fornecidas algumas funções logo no enunciado. Você notará que elas são familiares pois as implementamos nas últimas semanas ao longo dos laboratórios.

Python

```
'''Criando um ambiente com agente'''

import gymnasium as gym

def run():

    env = gym.make('FrozenLake-v1', map_name="8x8", is_slippery=True,
render_mode='human')

    obs = env.reset()[0]# obs = 0 a 63, sendo 0=top left e 63=bottom right

    terminated = False # True quando cai no buraco ou quando chega na recompensa

    truncated = False # True quando actions > 200

    while(not terminated and not truncated):

        action = env.action_space.sample()

        obs, reward, terminated, truncated, _ = env.step(action)

    env.close()

run()
```

Python

```
'''Plotar o gráfico de recompensas (episódios)'''

sum_rewards = np.zeros(epochs)

for t in range(epochs):
```

```
sum_rewards[t] = np.sum(rewards_per_episode[max(0, t-100):(t+1)])  
  
plt.plot(sum_rewards)  
  
plt.savefig('frozen_lake8x8.png')
```

Python

```
'''Salvando a Q-Table em arquivo para poder usá-la após o treinamento''  
  
f = open("frozen_lake8x8.pkl", "wb")  
  
pickle.dump(q, f)  
  
f.close()
```

Mais informações

Implementação do Frozen Lake usando Q-Learning segundo a biblioteca oficial do Gymnasium:
https://gymnasium.farama.org/tutorials/training_agents/frozenlake_q_learning/

Tutorial em vídeo de uma implementação do Frozen Lake com Q-Learning:
https://youtu.be/ZhoIgo3qqLU?si=_IpUn0FdKrq9OarX

Link da implementação de Frozen Lake usando Q-Learning no Jupyter Notebook:
<https://drive.google.com/file/d/1E9OUt42Jx7QuxTXmJnnuKZ3XtimV73AM/view?usp=sharing>